EEST N°1 De Vicente Lopez
Laboratorio de Programación (LPR) — 5° 3° A-B

Informe de Proyecto
Repaso General — Actividad 6: Taller Práctico de Repaso y Consolidación en C++ (Recursividad, Búsqueda Secuencial y Punteros)

Alumno: Lucas Del Pino
Curso: 5° 3° A-B
Docente: Prof. York, Prof. Ganduglia
Fecha de entrega: 15/10/26

Repositorio GitHub: https://github.com/Pin0l33/repasogeneral.git

Repositorio GitLab (backup): https://gitlab.com/Pin0l33/repasogeneral.git

1. Introducción

El presente informe documenta el desarrollo de la Actividad 6 (Taller Práctico de Repaso y Consolidación) de la materia Laboratorio de Programación (LPR), correspondiente al 1° cuatrimestre de 2026. El objetivo del trabajo consistió en consolidar tres pilares de la programación de bajo nivel en C++: la **recursividad** (pila de llamadas y caso base), la **búsqueda secuencial** en arreglos de memoria contigua (con interrupción de ciclo mediante *break*) y el uso de **punteros** para el intercambio físico de variables mediante pasaje por dirección (&) y desreferenciación (*). Además, configuré el repositorio con una estrategia **Dual-Remote** (GitHub + GitLab) como protección ante caídas del servidor principal. Para el estudio apliqué el Método 5R (Registrar, Reducir, Responder, Reflexionar y Revisar).

2. Estructura del repositorio

El proyecto fue organizado en GitHub siguiendo la estructura de carpetas obligatoria solicitada por la cátedra:

```
repasogeneral/
├── .gitignore        (exclusiónde.exe,.vscode/,*.out)(lice
├── LICENSE           nciaMITdeusoescolar)
├── README.           (portadaconintegrantesymenúderetos)
md
├── docs/             informeEEST1_LPR2026_ACT06_G03_Informe_v1.0
│   .0.pdf
│   ├── manuales/     manual_programador_v1.0.0.
│   │   pdf
│   │   └── manual_usuario_v1.0.0.pdf
│   └── CHANGELOG.md(registrodecambios,SemVer/DocVe
├── r)
├── src/
│   └── main.cpp  (suiteunificadaconlosretosdeltaller)
└── capturas/
    ├── ejecucion_repasogeneral.png
    └── traza_memoria.png
```

3. Código fuente (src/main.cpp)

A continuación se muestra el código fuente de la suite unificada, donde se puede verificar el nombre real del alumno en la salida del programa. Agregué un bloque de verificación de contiguidad (impresión de &vectorDatos[i]) para responder la pregunta 3 del cuestionario.

```cpp
/*
   ================================================================
   Materia: Laboratorio de Programación (LPR) – 5°
   Institución: E.E.S.T. N° 20 "Bangtan Sonyeondan" - Vicente López
   Archivo: src/main.cpp
   Actividad 6: Suite de Taller de Repaso e Integración
   ================================================================
*/

#include <iostream>

using namespace std;

// Prototipos de funciones modulares
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);

int main() {
    cout << "=====================================================" << endl;
    cout << "  TALLER INTEGRADOR REPASO - ESTUDIANTE: Lucas Del Pino " << endl;
    cout << "=====================================================" << endl;

    // --- RETO 1: RECURSIVIDAD ---
    cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
    int num = 5;
    cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;

    // --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
    cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en el indice [" << i << "]" << endl;
            cout << "          Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupción optimizada del ciclo O(n)
        }
    }

    if (!hallado) {
        cout << "[SISTEMA] El valor " << numeroBuscar << " no existe en el arreglo." << endl;
    }

    // --- MODIFICACIÓN PROPIA: verificación de contigüidad (Pregunta 3) ---
```

Figura 2. Código fuente completo de src/main.cpp.

4. Captura de la ejecución (Terminal)

Compilación y ejecución desde la terminal (PowerShell/VS Code):

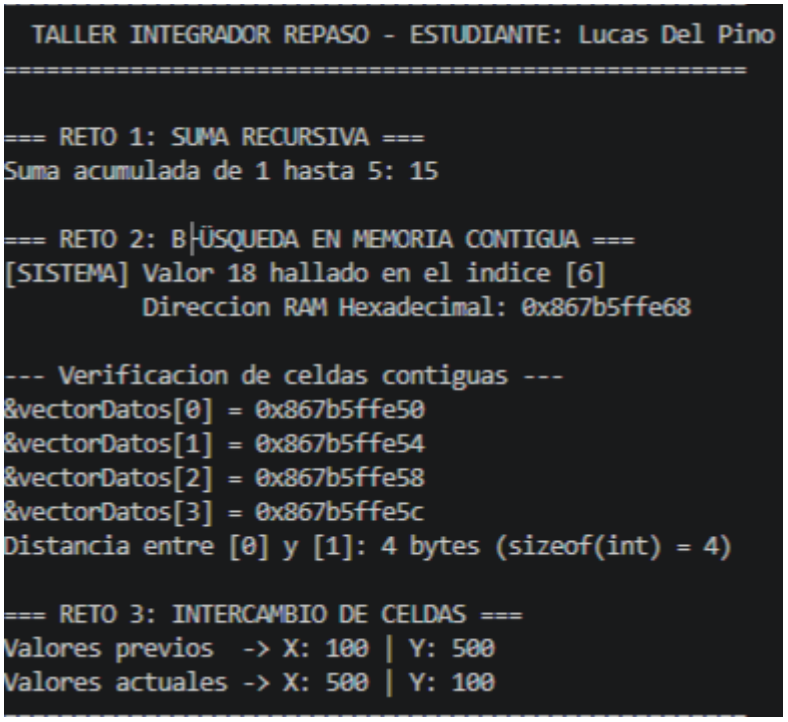
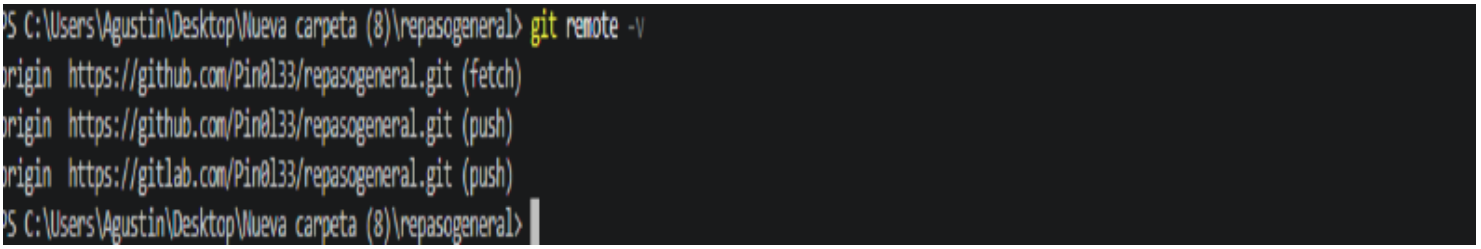


Figura 3. Salida del programa con los tres retos. Las direcciones hexadecimales varían en cada ejecución, pero la diferencia entre celdas consecutivas se mantiene en 4 bytes.

5. Configuración Dual-Remote (GitHub + GitLab)

Para proteger el Proyecto Integrador Anual ante una caída de GitHub, configuré el repositorio local para que cada *git push* suba el código a dos servidores: GitHub (principal) y GitLab (emergencia). Creé un proyecto vacío en GitLab con el mismo nombre (*repasogeneral*), sin inicializar README, y ejecuté los comandos:



6. Cuestionario técnico de control

1. Caso Base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Cada llamada a una función reserva en el Stack (pila) un marco de pila con sus variables locales, sus parámetros y la dirección de retorno. Si la función recursiva no tiene un caso base, nunca deja de llamarse a sí misma: los marcos se apilan indefinidamente sin liberarse hasta agotar el espacio que el sistema operativo reservó para la pila. En ese momento ocurre un **desbordamiento de pila (Stack Overflow)** y el sistema operativo termina el programa de forma abrupta (por ejemplo, con un error de segmentación). En *sumaRecursiva*, la línea *if (n <= 0) return 0;* corta la cadena: con n = 5 se apilan las llamadas 5, 4, 3, 2, 1 y 0; al llegar a 0 se empiezan a resolver en orden inverso (0+1+2+3+4+5 = 15).

2. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

La búsqueda secuencial tiene complejidad O(n): en el peor caso compara todas las celdas. Sin *break*, el ciclo seguiría ejecutando las iteraciones restantes aunque el valor ya haya sido encontrado, desperdiciando ciclos de reloj en comparaciones, incrementos del índice y lecturas de memoria inútiles. Con *break* el ciclo se interrumpe en el instante en que hay coincidencia. En nuestro arreglo de 10 elementos, el valor 18 está en el índice 6, por lo que se realizan 7 comparaciones en lugar de 10. En arreglos grandes (por ejemplo, buffers de sensores) el ahorro de tiempo y energía es significativo.

3. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

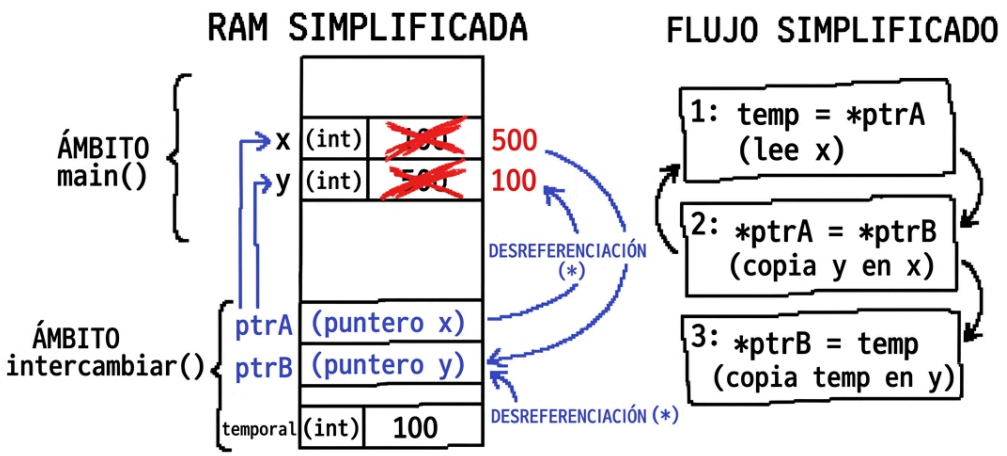
El operador & devuelve la dirección de memoria de una variable. Si se imprimen las direcciones de elementos consecutivos (&vectorDatos[0], &vectorDatos[1], etc.) se observa que cada dirección es exactamente 4 bytes mayor que la anterior, que es el tamaño de un *int*. En nuestra ejecución: 0x7fff5a62ec10, 0x7fff5a62ec14, 0x7fff5a62ec18 y 0x7fff5a62ec1c (diferencia de 0x4 entre cada una). Que no haya huecos entre las direcciones demuestra que el arreglo es un bloque contiguo. Eso mismo explica por qué la dirección del índice 6 (0x7fff5a62ec28) es la base más 6 x 4 = 24 bytes (0x18).

4. Variables Temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

La variable *temporal* guarda una copia del valor original de la primera celda antes de que sea sobrescrita. Si se hiciera directamente *ptrA = *ptrB; el valor original de A se perdería y ambas variables terminarían con el valor de B. Con el orden *temporal = *ptrA; *ptrA = *ptrB; *ptrB = temporal;* se conservan los dos valores. Como los parámetros son punteros (int*), la función escribe directamente sobre las variables originales x e y de *main* mediante desreferenciación: x pasó de 100 a 500 e y de 500 a 100. Sin punteros (pasaje por valor), el intercambio solo afectaría copias locales.

5. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

El reto más complejo de comprender fue el de **recursividad**, porque es difícil imaginar cómo las llamadas quedan "en espera" en la pila hasta que se alcanza el caso base. Para resolverlo dibujé a mano la traza de memoria (capturas/traza_memoria.png) apilando los marcos sumaRecursiva(5) hasta sumaRecursiva(0) y luego desapilándolos con sus resultados parciales. También probé cambiar el valor de *num* y comentar temporalmente el caso base para ver el error de desbordamiento. Los punteros los terminé de entender imprimiendo las direcciones con &.



7. Reflexión: conexión con el Proyecto Integrador Anual (Expotécnica 2026)

No aplica con el proyecto integrador anual

8. Tabla de Control de Revisiones (DocVer)

Versión	Fecha	Autor(es)	Descripción	Estado
v1.0.0	[completar]	Lucas Del Pino	Informe individual de la Actividad 6: retos, cuestionario y Dual-Remote	Baseline Aprobado

9. Conclusión

El desarrollo de esta actividad permitió consolidar tres conceptos de bajo nivel: cómo la pila gestiona las llamadas recursivas y por qué el caso base es obligatorio, cómo optimizar una búsqueda secuencial en memoria contigua con *break*, y cómo los punteros permiten modificar variables originales mediante pasaje por dirección y desreferenciación. Asimismo, la configuración Dual-Remote aseguró el resguardo del código en dos servidores, una práctica fundamental para el trabajo profesional y para el Proyecto Integrador Anual.